

Mini-Projects

Mini Project 1

Mini-Project 1 · Released Aug 14, 2026 · Due Sep 13, 2026

Implement your very own C-Shell & Implement the MLFQ scheduling policy for xv6.

Before you start

This is an individual assignment.

Deadlines

There are two submissions for this assignment. You are expected to submit parts [A](#) to [C](#) of C-Shell for the mid-submission, and remainder of the assignment for the end-submission.

- **Mid-submission:** August 24, 11:59 pm
- **End-submission:** September 13, 11:59 pm

You may use your late days for both of the submissions.

Submission

The exact details about submission will be made clear shortly.

However, you **NEED** to maintain a `git` repository for the duration of the project and commit iteratively as you go. Making a bunch of big commits right before the deadline may result in penalization.

And before you ask: you **DO NOT** need a GitHub repo to maintain a `git` repo. You can initialize one locally and keep it there until we figure out submission.

You can refer to the [resource on git](#) for more details.

Make sure your repository has the following structure:

```
mini-project1/
```

Copy

```
├── c-shell/
│   ├── src/
│   ├── include/
│   └── Makefile
├── xv6/
│   ├── user/
│   ├── mkfs/
│   ├── kernel/
│   ├── Makefile
│   ├── report.md    # Or report.pdf
└── README.md
```

Grading

You will be graded based on both your implementation and an in-person evaluation with a TA. You are expected to understand the code that you submit.

Doubt Doc

 Doubts

 Answers

All the best! Have fun :)

C Shell [200]

General Requirements

- Break the project down into multiple `.c` and `.h` files based on functionality. Monolithic code will be penalized.
- Use only C POSIX library headers and functions. You can use [this](#) for reference.
- Compile with the following flags for POSIX compliance:

```
gcc -std=c99 \
-D_POSIX_C_SOURCE=200809L \
-D_XOPEN_SOURCE=700 \
-Wall -Wextra -Werror \
-Wno-unused-parameter \
-fno-asm \
your_file.c
```

Copy

- Final build must compile with `make all` in the shell directory, producing `shell.out` in that same directory.

References

These could be helpful for you to understand things required to implement this section.

- [POSIX.1-2017 spec](#)
- `man 2` pages for `fork`, `exec`, `wait`, `pipe`, `dup2`, `open`. These cover most of what you'll need.
- Use the source of actual `bash` (will be helpful to understand how to do certain parts of the mini-project)

Part A: Shell Input [20]

This is the base for the rest of your project.

Banned syscalls for Part A: `exec*` (any syscall that starts with `exec`)

A1: Shell Prompt [3]

Your shell should show a prompt so the user knows it's ready for input.

```
<username@hostname:currentpath>
```

Copy

Requirements:

1. Display the prompt whenever the shell isn't running a foreground process.
2. The directory the shell is started in becomes its home directory.
3. If the current working directory has the home directory as an ancestor (e.g. `/path/to/home/osn`), replace the home directory prefix with `~`. So the example becomes `~/osn`.
4. If the current working directory does not have the home directory as an ancestor, show the absolute path as is.

Example

```
<goatam@iiit:~/osnmp1/> make
```

Copy

```
<goatam@iiiit:~/osnmp1/> ./shell.out  
<goatam@iiiit:~>
```

A2: User Input [2]

Requirements:

1. The shell should allow a user to type input.
2. On enter/return, the shell should consume the input.
3. After consuming input, the shell should display the prompt again.

Example

```
<goatam@iiiit:~> Hi there guys!  
<goatam@iiiit:~> This shell is cool!  
<goatam@iiiit:~>
```

[Copy](#)

A3: Input Parsing [15]

Your shell must scan and validate every line before executing anything. This section defines two layers: a lexer that turns characters into tokens, and a grammar that validates the token sequence. Validation must be performed.

Note: Since both Lexer and Parser follow a regular grammar, use your Automata Theory skills to the test.

Lexer: The lexer reads the raw input line and produces a sequence of tokens, some specifications are given below. Whitespace (space, tab, newline, carriage return) separates tokens and is otherwise discarded -it is never itself a token.

- **Character Classes:**

```
special    → | & > < ;  
quote      → " | '  
escape     → \  
space      → ' ' | \t | \n | \r  
ordinary   → any character that is none of the above
```

[Copy](#)

- **Tokens:**

```
OP_PIPE    → |
OP_AMP     → &
OP_SEMI    → ;
OP_LT      → <
OP_GT      → >
OP_GTGT    → >>
```

Copy

```
WORD       → fragment+
```

```
fragment   → ordinary
            | escape any_char
            | " dq_body "
            | ' sq_body '
```

```
dq_body     → ( escape any_char | any char except " and \ ) *
sq_body     → ( any char except ' ) *
```

- *Apply maximal munch*: i.e. at each position, take the longest match. e.g. `>>` is therefore one token, never two `>`.

Some details on Quoting:

Construct	Effect
On the unquoted value <code>\c</code> outside quotes	Contributes <code>c</code> literally; the backslash is removed
<code>"..."</code>	Contributes the body; <code>\"</code> becomes <code>"</code> , <code>\\</code> becomes <code>\</code> ; any other <code>\c</code> stays as both characters
<code>'...'</code>	Contributes the body verbatim; no escape processing, so <code>\</code> is literal
Special characters inside quotes or escaped	Lose their operator meaning entirely

Lexical errors: The lexer must reject and print `cshell: invalid syntax` when:

- A quote is opened and never closed before end of line.
- A `\` appears as the final character of the line.

Note: There is no line continuation and no multi-line input. Hint: use EOF to your advantage.

Grammar:

Terminals are the token classes above. Start symbol is `LINE`

```
LINE  →  ε
      |  WORD ARG

ARG   →  ε
      |  WORD   ARG
      |  OP_LT  TGT
      |  OP_GT  TGT
      |  OP_GTGT TGT
      |  OP_PIPE CMD
      |  OP_SEMI CMD
      |  OP_AMP  BG

CMD   →  WORD ARG

TGT   →  WORD ARG

BG    →  ε
      |  WORD ARG
```

Copy

Note: This is a right linear grammar, enjoy.

`TGT` is the name of a file or a redirection output, `CMD` is a valid command

Instructions:

- You **do not need** a full AST, any structure you find convenient (like a Linked List) works. Just make sure you are representing enough information internally to allow further sections to be implemented.
- You can use any type of parser and lexer, as long as it conforms to the grammar provided.

Requirements

- Verify whether an inputted command is valid or invalid per the grammar.
- If invalid, print `cshell: invalid syntax`.
- A line that is empty or contains only whitespace is valid: print a fresh prompt.
- Whitespace between tokens is ignored, in any amount. Whitespace inside quotes is preserved.

Examples:

```
<goatam@iiiit:~/osnmp1> cat meow.txt | meow; meow > meow.txt &
<goatam@iiiit:~/osnmp1> echo a & echo b &
<goatam@iiiit:~/osnmp1> cat < a.txt < b.txt > out.txt
<goatam@iiiit:~/osnmp1> echo "line one" > a.txt
<goatam@iiiit:~/osnmp1> echo "a > b"
<goatam@iiiit:~/osnmp1> cat my\ file.txt
<goatam@iiiit:~/osnmp1> echo ""
<goatam@iiiit:~/osnmp1> reveal -ta -aaa -tt
<goatam@iiiit:~/osnmp1> cat meow.txt | ; meow
cshell: invalid syntax
<goatam@iiiit:~/osnmp1> echo hi ;
cshell: invalid syntax
<goatam@iiiit:~/osnmp1> echo hi & &
cshell: invalid syntax
<goatam@iiiit:~/osnmp1> cat <
cshell: invalid syntax
<goatam@iiiit:~/osnmp1> cat << f
cshell: invalid syntax
<goatam@iiiit:~/osnmp1> | sort
cshell: invalid syntax
<goatam@iiiit:~/osnmp1> echo "unclosed
cshell: invalid syntax
<goatam@iiiit:~/osnmp1> echo trailing\
cshell: invalid syntax
<goatam@iiiit:~/osnmp1>
```

[Copy](#)

Part B: Shell Intrinsics [40]

These are commands any shell worth its salt supports.

Banned syscalls for Part B: `exec*` (any syscall that starts with exec)

B1: hop [10]

Syntax: `hop ((~ | . | .. | - | name)*)?`

Purpose: The hop command changes the shell's current working directory. Unlike a plain cd, hop falls back to a frequency lookup when a name doesn't resolve directly, so you can jump to directories you visit often without typing the full path.

Requirements

Execute the following sequentially for each passed argument:

1. "~" or no arguments: change the CWD to the shell's home directory.
2. ".": do nothing, stay in the same CWD.
3. "..": change the CWD to the parent directory of the CWD, or do nothing if the CWD has no parent.
4. "-": change the CWD to the previous CWD, or do nothing if there was no previous CWD.
5. "name": if this resolves to a valid relative or absolute path from the CWD, change to it directly.
6. If "name" does not resolve directly, fall back to a frequency lookup as defined below.
7. If neither a direct match nor a frequency match is found, output `hop: no such directory`

Frequency

Keep a persistent record of directories hopped into, across shell sessions.

1. Every successful hop updates that directory's standing.
2. Standing depends on both frequency and recency of visits.
3. When "name" doesn't resolve directly, hop into the highest ranked directory whose path contains "name" as a substring.
4. If the top match no longer exists on disk, skip it and try the next best.

Scoring, decay, and storage are up to you, but must be deterministic.

Resource: same idea behind zoxide's `cd` replacement:

<https://github.com/ajeetsouza/zoxide/wiki/Algorithm>

Example

```
<goatam@iiit:~/osnmp1> hop ~
<goatam@iiit:~> hop ..
<goatam@iiit:/home/> hop goatam/osnmp1 .. -
<goatam@iiit:~/osnmp1> hop osn
<goatam@iiit:~/osnmp1>
```

Copy

B2: reveal [10]

Syntax: `reveal (-(a|t)*)* (~ | . | .. | - | name)?`

Purpose: The reveal command lists the files and directories in a directory.

Requirements

1. If no argument is passed, `reveal` lists the contents of the current working directory.
2. “a”: reveal all files and directories, including hidden ones (starting with `.`). Default is to hide them.
3. “t”: recursively reveal the contents of all subdirectories. the content of the subdirectory must be displayed after the directory itself (refer example below)
4. When both flags are set, recursively show all files, hidden included, in the format given below.
5. When neither flag is set, print entries in the format of `ls`.
6. The argument passed resolves the same way as hop’s `~`, `.`, `..`, `-`, and direct path cases, except reveal lists directory contents instead of changing the CWD. There is no frequency fallback: if the argument does not resolve to an existing directory, print `reveal: no such directory`.
7. Entries are always listed in lexicographic order, sorted by ASCII value.
8. If reveal is passed too many arguments, print: `reveal: invalid syntax`
9. If `reveal -` is run before any hop has been run since shell start, print: `reveal: no such directory`

Example

```
<goatam@iiit:~> reveal ~
osnmp1
<goatam@iiit:~> hop ..
<goatam@iiit:/home> reveal
goatam
<goatam@iiit:/home> hop goatam/osnmp1
<goatam@iiit~/osnmp1> reveal -t
Makefile
README.md
include/
include/parser.h
include/shell.h
shell.out
src/
src/main.c
```

Copy

```
src/parser.c
<goatam@iiit:~/osnmp1> reveal -ta
Makefile
README.md
include/
include/parser.h
include/shell.h
shell.out
src/
src/main.c
src/parser.c
<goatam@iiit:~/osnmp1> reveal -ta -ttttttaaaaaaaaaaatttt -aaaa -tttt
Makefile
README.md
include/
include/parser.h
include/shell.h
shell.out
src/
src/main.c
src/parser.c
```

B3: peek [10]

Syntax `peek (-(n|r)*)* filename*`

Purpose: The peek command concatenates the contents of one or more given files (or standard input) and then prints it to the standard output.

Requirements:

1. “n”: counts the number of non-empty lines in the input. The line number must be printed before the corresponding line. “r”: print the contents of the input in reverse line order. For seekable input (regular files), use lseek to read the file backwards in fixed-size chunks rather than loading it all into memory at once. For non-seekable input (pipes, standard input), buffering the full input before reversing is allowed.
2. When both flags are set, print the contents of the file in reverse order, with its corresponding line number.
3. When no flag is set, print the contents of the file as is.
4. If more than one file is provided, concatenate their contents in the given order.
5. When -r is combined with multiple files, each file’s lines are reversed independently before concatenation; the overall file order given on the command line is preserved.

6. If no filename is provided, or when filename is -, read from standard input.
7. If filename does not exist, print `peek: no such file or directory`
8. If filename is a directory, print `peek: is a directory`

Example

```
<goatam@iiit~/osnmp1> cat README.md
# OSN MP1

This is OSN shell assignment.

<goatam@iiit~/osnmp1> peek -n README.md
1 #OSN MP1

2 This is OSN shell assignment
<goatam@iiit~/osnmp1> peek -r README.md
This is OSN shell assignment

# OSN MP1
<goatam@iiit~/osnmp1> peek -rn -rrrrrrrrrr -rrrrrr README.md
2 This is OSN shell assignment

1 #OSN MP1
<goatam@iiit~/osnmp1> peek -
meow
meow
<goatam@iiit~/osnmp1> peek README.md Makefile
#OSN MP1

This is OSN shell assignment.
CC = gcc
CSTD = -std=c99

Preprocessor definitions
DEFINES = -D_POSIX_C_SOURCE=200809L
-D_XOPEN_SOURCE=700
```

Copy

B4: locate [10]

Syntax: `locate filename*`

Purpose: The `locate` command returns the pathnames of the files (or links) which would be executed in the current environment, had its arguments been given as commands in a strictly

POSIX-conformant shell.

Requirements:

1. For each filename, first check the current working directory. If a match exists there, print its absolute path first. Then search each directory listed in `PATH`, in order, and print the absolute path of every match found. This means a name matching in both the cwd and two `PATH` entries prints three lines, in that order.
2. The absolute path to the executable must be printed (even if the executable exists in the current working directory).
3. If multiple arguments are provided, print the path for each one of them, in the order specified by the user.
4. If multiple matching executable paths are found, print all the paths.
5. If `filename` is not a valid executable, print: `locate: command not found (commandname)` and continue processing the remaining arguments.
6. You may assume that file paths will not be passed as arguments.

Example:

```
<goatam@iiiit:~> locate java
/usr/bin/java
<goatam@iiiit:~> locate man java python nada
/usr/bin/man
/usr/bin/java
/usr/bin/python
locate: command not found (nada)
<goatam@iiiit:~> locate sudo
/usr/bin/sudo
/bin/sudo
```

Copy

Part C: File Redirection and Pipes [50]

For this part, when processing commands with sequential (`;`) or background (`&`) operators, you should only execute the first command group and ignore the rest.

C1: Command Execution [8]

You must allow execution of arbitrary commands, found either as a relative/absolute path to an executable, or by name.

Requirements

1. If the command name contains a `/`, treat it as a literal path and execute it directly if it exists and is executable.
2. If the command name does not contain a `/`, first check whether an executable with that name exists in the current working directory, and run it if so.
3. If no such executable exists in the current working directory, search `PATH` as usual.
4. To skip the current directory check and search `PATH` directly, prefix the command name with `%`. `%name` must always resolve via `PATH`, even if a matching executable exists in the current directory.
5. If no executable is found by any of the above, output `cshell: command not found (commandname)`.

Example

```
<goatam@iiit:~/proj> echo hello
hello
<goatam@iiit:~/proj> ls
build.sh
<goatam@iiit:~/proj> build.sh
running local build script
<goatam@iiit:~/proj> %build.sh
cshell: command not found (build.sh)
<goatam@iiit:~> dosakdaoskdos
cshell: command not found (dosakdaoskdos)
```

Copy

C2: Input Redirection [12]

Syntax: `command < filename`

Purpose: The input redirection operator allows a command to read its standard input from one or more files instead of the terminal.

Requirements

1. The shell must open each specified file for reading using the `open()` system call with `O_RDONLY` flag.
2. If any file does not exist or cannot be opened, the shell must print `cshell: no such file or directory` and not execute the command.
3. A command may have more than one input redirection (e.g. `command < f1 < f2`). The

command's standard input must consist of the contents of every listed file, read in the order given, as one continuous stream.

4. The shell must redirect the command's standard input (`STDIN_FILENO`) to this stream using `dup2()` .
5. The shell must close any file descriptors it opens once they are no longer needed.

Example

```
<goatam@iiiit:~> cat < notes.txt
meeting at 5pm
<goatam@iiiit:~> echo "line one" > a.txt
<goatam@iiiit:~> echo "line two" > b.txt
<goatam@iiiit:~> cat < a.txt < b.txt
line one
line two
<goatam@iiiit:~> cat < missing.txt
cshell: no such file or directory
```

Copy

C3: Output Redirection [12]

Syntax: `command > filename` or `command >> filename`

Purpose: The output redirection operators allow a command's output to be written to one or more files instead of the terminal.

Requirements

1. A command may have more than one output redirection (e.g. `command > f1 >> f2 > f3`). Every listed file must receive the complete output of the command.
2. Each redirection keeps its own mode independently: `>` truncates its file (or creates it with permissions `0644` if it doesn't exist), `>>` appends to its file (or creates it with `0644` if it doesn't exist).
3. When output redirection is present, the command's output must not appear on the terminal, only in the listed files.
4. If any listed file cannot be opened for writing, the shell must print `cshell: unable to create file for writing` and not execute the command.
5. Input and output redirection must work together (e.g. `command < input.txt > output.txt >> log.txt`).

Example

```
<goatam@iiit:~> echo hi > out1.txt > out2.txt
<goatam@iiit:~> cat out1.txt
hi
<goatam@iiit:~> cat out2.txt
hi
<goatam@iiit:~> echo again >> out1.txt > out3.txt
<goatam@iiit:~> cat out1.txt
hi
again
<goatam@iiit:~> cat out3.txt
again
```

Copy

C4: Command Piping [18]

Syntax: `command1 | command2 | ... | commandN`

Purpose: The pipe operator connects one command's output to the next command's input.

Requirements

1. The shell must create pipes using `pipe()` for each `|` in the command.
2. For each command in the pipeline, the shell must fork a child process.
3. The shell must redirect just the standard output of `command[i]` to the write end of `pipe[i]`.
4. The shell must redirect the standard input of `command[i+1]` to the read end of `pipe[i]`.
5. The parent shell must close every pipe file descriptor it holds after forking.
6. Each child must close any pipe file descriptors it does not use before executing its command.
7. The parent shell must wait for all commands in the pipeline to complete before continuing.
8. If a command in the pipeline fails to execute, the shell prints `cshell: command not found (commandname)` for that stage, and the rest of the pipeline still runs.
9. File redirection and pipes must work together (e.g. `command1 < input.txt | command2 > output.txt`).

Example

```
<goatam@iiit:~> printf "banana\napple" | sort
```

Copy

```
apple
banana
<goatam@iiit:~> badcmd | sort
cshell: command not found (badcmd)
<goatam@iiit:~> cat < in.txt | sort > out.txt
```

Part D: Sequential and Background Execution [25]

D1: Sequential Execution [10]

Syntax: `command1 ; command2 ; ... ; commandN`

Purpose: The semicolon operator allows an arbitrary number of commands to execute one after another.

Requirements:

1. The shell must execute each command in the order they appear.
2. The shell must wait for each command to complete before starting the next.
3. If a command fails to execute, the shell must print `cshell: command not found (commandname)` and then stop executing the remaining commands in sequence.
4. Each command in the sequence must be treated as a complete `shell_cmd` as defined in the grammar.
5. The shell prompt must be displayed either after all the commands completed successfully, or because one command has failed.

```
<goatam@iiit:~> echo hello ; echo world
hello
world
<goatam@iiit:~> echo hello ; nada ; echo world
hello
cshell: command not found (nada)
```

Copy

D2: Background Execution [15]

Syntax: `command1 & command2 & ... & commandN &`

Purpose: The ampersand operator allows the command preceding it to run in the background while the shell continues to accept new commands or execute the remaining commands.

Requirements:

1. When a command ends with `&`, the shell must fork a child process and execute it in the background without waiting for it to complete.
2. When multiple commands are separated by `&`, each command must be launched independently in the order in which it appears.
3. The shell must print the background job number and process ID in the format:
`[job_number] process_id`
4. Job numbers are assigned session-wide and increase monotonically. They are never reused, even after a job completes.
5. The shell must immediately print a new prompt after launching all the background processes.
6. The shell must install a `SIGCHLD` handler to detect when a background process completes.
7. The `SIGCHLD` handler must reap terminated processes using `waitpid()` with `WNOHANG`, so that the shell never blocks while handling background processes.
8. When a background process completes successfully, the shell must print: `command with pid process_id exited normally`.
9. When a background process exits abnormally, the shell must print: `command with pid process_id exited abnormally`.
10. As soon as a background process terminates, the shell must report its completion, including when waiting for user input.
11. If any foreground process is being executed while a background process terminates, its completion must be reported after the foreground process terminates.
12. Background processes must not have access to the terminal for input.
13. For a pipeline, the reported pid should be the pid of the first command in the pipeline.

Note: You may have to do the first part of **E1** to be able to implement functionality **#12** of this section.

Example

```
<goatam@iiiit:~> echo hello & sleep 10 &  
hello  
[1] 4021  
[2] 4022  
echo with pid 4021 exited normally  
<goatam@iiiit:~>
```

Copy

```
sleep with pid 4022 exited normally
<goatam@iiiit:~> sleep 1 & sleep 2 & cat
[1] 4023
[2] 4024
hello
hello^C

sleep with pid 4023 exited normally
```

Part E: Exotic Shell Instrinsics [45]

E1: activities [10]

Syntax: `activities`

Purpose: The `activities` command lists every process the shell has spawned that is still running, grouped by process group.

Requirements

1. Every pipeline the shell launches must run in its own process group, using `setpgid()` on each child right after `fork()` in the parent, and also in every child before `execve`.
2. A standalone command (no `|`) counts as a process group of one.
3. `activities` must print one line per group, followed by one line per process in that group, indented under it.
4. Each process line must show: `pid`, command name, and state (`Running` or `Stopped`).
5. Print format:
 - Group line: `[job_number] pgid pgid_value`
 - Process line: `pid command_name state` (two-space indent, fields space-separated; exact column alignment does not matter)
6. Groups must be printed in the order they were launched, oldest first.
7. Processes that have exited must be removed from the list before printing.
8. For a pipeline, consider the pid of the first command in the pipeline.

Example

```
<goatam@iiiit:~> sleep 100 &
[1] 4021
```

Copy

```
<goatam@iiiit:~> cat | sort &
[2] 4030
<goatam@iiiit:~> activities
[1] pgid 4021
    4021 sleep    Running
[2] pgid 4030
    4030 cat      Running
    4031 sort     Running
```

E2: Terminal Control [15]

Purpose: Ctrl-C, Ctrl-Z, and Ctrl-D let the user interrupt, stop, or end shell sessions the way any real shell supports.

Requirements

1. The shell must ignore `SIGINT` and `SIGTSTP` and `SIGTTOU`.
2. Before running a foreground pipeline, the shell must give it the terminal via `tcsetpgrp()`.
3. After a foreground pipeline finishes or stops, the shell must reclaim the terminal via `tcsetpgrp()`.
4. Background jobs must never receive the terminal, so Ctrl-C and Ctrl-Z only affect the foreground pipeline.
5. On Ctrl-Z, the shell must wait using `WUNTRACED`, mark the pipeline's process group as `Stopped`, and print `[job_number] + Stopped command`, then return to the prompt.
6. On Ctrl-D at the prompt (EOF on stdin), the shell must exit.
7. If any job is Stopped when Ctrl-D is pressed, the shell must print `cshell: there are stopped jobs` and not exit.
8. If Ctrl-D is pressed again immediately afterward (no other input in between), the shell must exit.
9. When the shell exits (via Ctrl-D or otherwise) while background or stopped jobs still exist, the shell must send `SIGHUP` to every tracked job's process group before exiting. The shell must not wait for these processes to terminate.

Example

```
<goatam@iiiit:~> sleep 100
^C
<goatam@iiiit:~> sleep 100
^Z
```

Copy

```
[1] + Stopped      sleep 100
<goatam@iiiit:~> sleep 100 &
[2] 4021
<goatam@iiiit:~> ^C
<goatam@iiiit:~>
```

E3: resume [12]

Syntax: `resume %job_number (fg [--timeout <seconds>] | bg)`

Purpose: resume continues a stopped or backgrounded job, either bringing it to the foreground or letting it keep running in the background.

Requirements

1. `job_number` refers to a job as listed by `activities`.
2. The shell must send `SIGCONT` to the job's process group.
3. `fg`: the shell must give the job the terminal via `tcsetpgrp()`, mark it `Running`, wait for it to finish or stop again, then reclaim the terminal.
4. `bg`: the shell must mark the job `Running` and return to the prompt immediately, without waiting or touching the terminal.
5. If `--timeout <seconds>` is given with `fg`, the shell must start a timer using `alarm()` or `setitimer()` before waiting on the job.
6. If the job is still running when the timeout elapses, the shell must send it `SIGTERM`, print `resume: job timed out`, and reclaim the terminal.
7. If the job finishes or stops on its own before the timeout, the shell must cancel the pending timer.
8. If `job_number` does not correspond to a known job, print `resume: no such job`.
9. If the syntax does not match (missing job number, missing/invalid `fg` / `bg`, `--timeout` given without a valid number), print `resume: invalid syntax`.
10. `bg`: after marking the job `Running`, print `[job_number] + Running command`.
11. `fg`: before waiting on the job, print the job's command line (e.g. `sleep 100`), same as a normal foreground launch would.

Example

```
<goatam@iiiit:~> sleep 100
^Z
```

Copy

```

[1] + Stopped      sleep 100
<goatam@iit:~> resume %1 bg
[1] + Running      sleep 100
<goatam@iit:~> resume %1 fg
sleep 100
<goatam@iit:~> sleep 100
^Z
[1] + Stopped      sleep 100
<goatam@iit:~> resume %1 fg --timeout 3
sleep 100
resume: job timed out
<goatam@iit:~>

```

E4: ping [8]

Syntax: `ping <target> <signal_number>`

Purpose: The ping command sends a signal to a process or an entire process group.

Requirements

1. `<target>` is a plain number, interpreted as a pid, unless prefixed with `%`, in which case it is interpreted as a job number from `activities` and the signal is sent to every process in that job's group.
2. The signal actually sent is `signal_number % 64`.
3. `signal_number` is validated before `<target>` is looked up. If it fails validation (requirement 6), the syntax error is printed regardless of whether `<target>` would have resolved.
4. If `<target>` does not correspond to any known process or job, print `ping: no such process found`.
5. On success, print `Sent signal signal_number to <target>`.
6. The message always echoes the original typed `signal_number`, not the value after `% 64`.
7. If `signal_number` is not a valid non-negative integer, print `ping: invalid syntax`. Negative numbers are invalid syntax, not a valid signal value to be reduced by the modulo.
8. `ping` may only target pids or jobs the shell itself has spawned and is still tracking. A pid that exists on the system but wasn't launched by this shell must be treated as unknown and print `ping: no such process found`.

Example

```
<goatam@iiit:~> cat | sort &
[2] 4030
<goatam@iiit:~> ping 4030 15
Sent signal 15 to 4030
<goatam@iiit:~> ping %2 9
Sent signal 9 to %2
<goatam@iiit:~> ping 4030 79
Sent signal 79 to 4030
<goatam@iiit:~> ping 99999 9
ping: no such process found
<goatam@iiit:~> ping %5 9
ping: no such process found
<goatam@iiit:~> ping 4030 abc
ping: invalid syntax
<goatam@iiit:~> ping 4030 -3
ping: invalid syntax
```

Part F: Fun Stuff [20]

F1: spy [10]

Syntax: `spy [pid]`

Purpose: `spy` lists information about open files associated with a given process to the standard output. An open file may be a regular file, a directory, a block special file, a character special file, an executing text reference, a library or a stream.

Requirements:

1. If no process ID is provided by the user, display the open files of the running shell.
2. If a process ID is provided, display the open files of the specified process.
3. For each entry, display: `PID` , `FD` , `TYPE` , `PATH` .
4. `FD` should display the following: `cwd` : current working directory; `mem` : memory-mapped file (note that each unique memory mapped file should be printed only once); `txt` : executable file; numeric file descriptors such as `0` , `1` , `2` when they are open. Refer the example below.
5. The `TYPE` field must correctly identify the file type of the object associated with each entry (e.g. `REG` , `DIR` , `CHR` , etc). Note that network specific file type objects are out of the scope of this mini-project.
6. `PATH` must contain the path associated with the corresponding open object represented by

FD .

7. If the given process ID does not correspond to an existing process, print: `spy: no such process` .
8. If more than one process IDs are passed as arguments, print: `spy: invalid syntax` .

Hint: Process information can be accessed at `/proc` filesystem. Explore the directories in it to obtain the corresponding information. Think about how you can use `readlink()` for `PATH` .

`lsof` source code may be a useful resource for understanding how such information can be obtained.

Example:

```
<goatam@iiiit:~> spy 21
PID    FD    TYPE  PATH
21     cwd   DIR   /home/goatam
21     txt   REG   /usr/bin/sleep
21     mem   REG   /usr/lib/locale/locale-archive
21     mem   REG   /usr/lib/libc.so.6
21     mem   REG   /usr/lib/gconv/gconv-modules.cache
21     mem   REG   /etc/ld.so.cache
21     mem   REG   /usr/lib/ld-linux-x86-64.so.2
21     0     CHR   /dev/pts/0
```

Copy

F2: snoop [10]

Syntax: `snoop command [args...]` or `snoop -p pid`

Purpose: snoop traces a process's syscalls and reports a summary once it exits.

Requirements

1. If a command is given, the shell must fork, call `PTTRACE_TRACEME` in the child, then `execve` the command.
2. If `-p pid` is given, the shell must attach to the running process using `PTTRACE_ATTACH` .
3. Use `PTTRACE_SYSCALL` to stop the tracee at each syscall entry and exit.
4. Record the syscall number and a timestamp at entry, and a timestamp at exit, to compute time spent per call.
5. Once the traced process exits, print a summary table: syscall name, number of calls, total

time spent in it.

6. Sort the summary by call count descending. Break ties by order of first occurrence.
7. Syscall numbers not in your lookup table must print as `syscall_N`, where N is the syscall number.
8. If the given pid does not exist, print `snoop: no such process`
9. If the command to launch does not exist, print `snoop: command not found`

Example

```
<goatam@iiiit:~> snoop sleep 1
syscall      calls    time
nanosleep    1         1.000s
write         1         0.000s
exit_group   1         0.000s
<goatam@iiiit:~> snoop -p 99999
snoop: no such process
```

Copy

Conclusion

yay! you have a cshell now!

Implementing a Multi-Level Feedback Queue (MLFQ) Scheduler in xv6

The default scheduling policy in xv6 is round-robin (RR). In this mini-project, you will implement an additional scheduling policy, the **Multi-Level Feedback Queue (MLFQ)**, and integrate it into the xv6 kernel.

The kernel must use **only one** scheduling policy, selected **at compile time**. If no policy is specified, the kernel must default to the original round-robin scheduler.

We expect you to have finished the DIY section of **Homework 2** (FIFO scheduler for xv6) before attempting this part of the mini project.

1. Build System Changes

Modify the `Makefile` to support a `SCHEDULER` macro that selects the scheduling algorithm to compile in.

Supported flag:

Flag	Scheduler
<code>MLFQ</code>	Multi-Level Feedback Queue

If `SCHEDULER` is not passed, the build must default to plain round-robin.

Example build commands:

```
# Default (Round Robin)
make clean; make qemu

# MLFQ
make clean; make qemu SCHEDULER=MLFQ
```

Copy

An unset `SCHEDULER` must compile the original RR code path unchanged.

2.Scheduling Specification [100]

2.1 MLFQ [70]

2.1.1 QUEUES

Implement **four priority queues**, numbered 0 (highest) to 3 (lowest).

2.1.2 TIME SLICES

Priority	Time Slice (timer ticks)
0	1
1	4
2	8
3	16

A “tick” refers to a clock/timer interrupt (see `kernel/trap.c`).

2.1.3 SCHEDULING RULES

1. **New process placement:** On creation, a process is pushed to the end of queue 0.
2. **Strict priority selection:** The scheduler always runs a process from the highest-priority non-empty queue.
 - Example: a process running in queue 2 (with queues 0 and 1 empty) must be **preempted** if a new process arrives in queue 0. Preemption only needs to occur at tick boundaries — i.e., when the kernel next regains control of the CPU (end of the current tick).
3. **Process completion:** When a process exits, it is removed from the queuing system.
4. **Time-slice exhaustion:** If a process uses its **entire** time slice for its current queue, it is preempted and moved to the **end of the next lower queue**. If it is already in queue 3 (the lowest), it is re-inserted at the end of **queue 3**.
5. **Voluntary yield (e.g., I/O):** If a process voluntarily gives up the CPU before its slice expires, it leaves the queuing network. When it becomes runnable again, it is inserted at the **tail of the same queue** it was in when it yielded (its priority is unchanged).
6. **Lowest queue behavior:** Queue 3 is scheduled **round-robin**.
7. **Priority boosting (anti-starvation):** Every **48 ticks**, all processes in the system are moved to queue 0, regardless of their current queue.

2.1.4. DEBUGGING SUPPORT: `PROCDUMP`

`kernel/proc.c` contains `procdump()`, which is invoked when the user presses **Ctrl+P** on the console.

- Extend `procdump()` to print, for each process: its PID, name, state, **current priority/queue number**, and any other bookkeeping data relevant to verifying scheduler correctness (e.g. ticks consumed in current slice, ticks since last boost).
- Use this output while developing to confirm processes are moving between queues, preempting, and being boosted as specified.
- You may add any helper printing/logging code needed for testing; this is not restricted to the exact format already in `procdump`.

2.2. Cross-Scheduler Comparison [20]

Compare **FIFO**, **Round Robin (RR)**, and **MLFQ** using the following metrics, computed over the same fixed workload/process set:

Metric	Meaning	Goal
--------	---------	------

Turnaround Time	Time from process arrival to completion	Minimize ↓
Waiting Time	Total time spent waiting in the ready queue	Minimize ↓
Response Time	Time from arrival until the process first gets the CPU	Minimize ↓

2.3 Report Requirements [10]

Submit a short report (PDF or Markdown) containing:

2.3.1 IMPLEMENTATION SUMMARY

For **each** specification item above (Makefile/ `SCHEDULER` macro, `struct proc` changes, `allocproc()` changes, queue selection/preemption logic, time-slice handling, voluntary yield handling, priority boosting, `procdump` changes), write a **few sentences** explaining what you changed and why.

2.3.2 MLFQ ANALYSIS

`schedulertest` is not part of default xv6: you will need to write it yourself as a custom user-space test program that spawns multiple child processes to exercise your scheduler. Use it to create several test processes that consume the CPU for **varying** amounts of time before **voluntarily** giving it up (e.g. simulate different CPU-burst lengths).

Produce a **timeline / scatter plot**:

- **X-axis:** time elapsed (in ticks) since scheduler start.
- **Y-axis:** queue ID (0–3) the process is currently in.
- **Color-coded by process** (one color per PID/process).
- The plot must clearly show processes moving between queues over time **and** the periodic priority boost (all active processes jumping back to queue 0 every 48 ticks).

Include the graph and 3–5 sentences interpreting it (e.g., which processes behave as CPU-bound vs. I/O-bound, and how the boost affects them).

2.3.3 COMPARISON RESULTS

- A table (or bar chart) reporting average Turnaround Time, Waiting Time, and Response Time for each of the three schedulers, on an identical set of test processes.
- A short discussion (5–8 sentences) on the trade-offs observed — e.g., why MLFQ may have lower response time than FIFO, why RR's waiting time depends on quantum size, etc.